Collapse-Aware Maintenance of Encrypted Traffic Classifiers under Temporal Drift

Junwei Xiao, Xiaobo Jin, Xiucheng Wu, Zhiquan Liu, and Dongjie Liu

***Abstract*—Traffic-based inference supports device management and security monitoring in Internet of Things (IoT) networks, where evolving traffic calls for reliable classifier maintenance. We examine longitudinal encrypted application traffic, where aggregate metrics can conceal severe class-specific failures. We formalize absorber collapse, a class-asymmetric failure in which dominant classes absorb predictions from victim classes and drive their recall toward zero. On CESNET-TLS-Year22, 12 of 178 classes collapse within nine months, while five tested traffic-adapted test-time adaptation methods leave collapse-class F1 below 0.031. We propose Collapse-Aware Repair (CARE), which combines unsupervised class-health monitoring with head-only repair using newly labeled target samples, all-class reference replay, and knowledge distillation. With Batch Active Learning by Diverse Gradient Embeddings (BADGE), CARE raises macro-F1 from 0.629 to 0.700 and collapse-class F1 from 0.028 to 0.332 using 1,000 target labels under strict query-excluded evaluation over five repair seeds, while retaining stable-class F1 near the static level (0.894 versus 0.903). A QUIC stress test probes cross-protocol transfer. A separate 11-household IoT device-type holdout shows low-recall-class recovery alongside heterogeneous gains and losses across homes, underscoring the need to review candidate updates at the class level.**

***Index Terms*—Encrypted traffic classification, IoT traffic analytics, concept drift, class collapse, label-efficient maintenance, active learning.**

I. Introduction

Traffic-based inference supports device inventory and network monitoring in IoT environments even when payloads are unavailable. Residential studies document temporal and cross-home drift in device-type inference [1], [2], [3]. A central maintenance question extends across traffic-classification tasks: can aggregate accuracy conceal the failure of individual classes, and can a small number of fresh labels repair them without damaging the others? We investigate this question primarily with longitudinal encrypted application traffic and examine its reach in a separate 11-household IoT device-type setting.

Encrypted-traffic classifiers degrade as applications, protocols, and infrastructure evolve [4], [5]. We identify a class-asymmetric pattern, termed *absorber collapse*, in which a dominant predicted class attracts traffic from another class until the latter has near-zero recall. A model can therefore retain acceptable macro-F1 while no longer recognizing a particular class. Such failures matter when class predictions inform network monitoring or policy. High-confidence errors are difficult to correct with entropy-based test-time adaptation (TTA) or pseudo-labeling: both can reinforce the wrong absorber prediction.

We propose CARE (Collapse-Aware REpair), an operator-reviewed maintenance workflow. An unlabeled class-health monitor ranks potential failures; after review, Margin or BADGE selects target flows for labeling. A frozen encoder and a fine-tuned classification head use those labels together with all-class reference replay and source-head distillation. The repair is evaluated for both collapsed-class recovery and stable-class preservation. Figure 2 places this workflow in a possible IoT traffic-analytics system; the household experiment tests its repair stage, not a deployed gateway.

Our contributions are threefold: (1) a longitudinal characterization of absorber collapse and its empirical signatures; (2) a label-efficient, class-aware repair procedure with explicit preservation of unaffected classes; and (3) strict query-excluded evaluation across time, adapted baselines, a QUIC stress test, and a separate 11-home device-type study that reveals the limits of cross-household transfer. The latter uses a different label space and training protocol; its mixed outcomes are reported as an external stress test rather than as universal IoT efficacy.

II. Related Work

Temporal and cross-home shifts in IoT device traffic motivate model selection, class-prior adaptation, and budget-aware labeling [1], [2], [3]. Longitudinal encrypted-traffic studies likewise document changing class behavior [4], [5], [6]. Our focus is the narrower failure in which an existing class becomes nearly unrecognizable while aggregate performance can remain usable.

Label-free TTA methods such as Tent, EATA, SAR, CoTTA, and NOTE [7], [8], [9], [10], [11] adapt to distribution shift, but confident absorber errors provide little entropy-based corrective signal. Self-evolving traffic adaptation [12] uses high-confidence pseudo-labels; its collapse-specific recovery was not reported. Domain alignment [13], [14] and continual-learning methods [15], [16], [17] address broader adaptation or forgetting, while our fixed-label-space task requires recovery and preservation together.

Active learning allocates scarce labels through uncertainty or diversity [18], [19], [20]. Traffic-oriented drift and active-learning studies [21], [22], [23] address adjacent tasks. CARE

Junwei Xiao, Zhiquan Liu, and Dongjie Liu are with the College of Cyber Security, Jinan University, Guangzhou 511443, China (e-mail: xjw20528@stu2024.jnu.edu.cn; zqliu@vip.qq.com; djliu@jnu.edu.cn).

Xiaobo Jin is with the Department of Intelligent Science, School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou 215123, China (e-mail: Xiaobo.Jin@xjtlu.edu.cn).

Xiucheng Wu is with Coremail, Guangzhou 510006, China (e-mail: mwu@coremail.cn).

Dongjie Liu is the corresponding author.

combines class-level monitoring, target-label acquisition, and reference-based head repair; we compare its components and adapted baselines under stated budgets and protocols.

III. Empirical Analysis: Class Collapse Under Drift

A. Problem Formulation

Let $f_\phi : \mathcal{X} \to \mathbb{R}^d$ denote a frozen feature encoder and $h_\theta : \mathbb{R}^d \to \mathbb{R}^C$ a classification head, jointly trained on a source period $\mathcal{D}^{\text{src}}$ with C application classes. At a later target period t, the data distribution shifts: $P_t(\mathbf{x}, y) \neq P_{\text{src}}(\mathbf{x}, y)$. We define a class c as *collapsed* at period t if its recall $R_c^{(t)} = \text{TP}_c/(\text{TP}_c + \text{FN}_c) < \tau$, where TP and FN denote true positives and false negatives; we use τ=0.1. A collapsed class c has a *primary absorber* $a(c) = \arg\max_{j \neq c} n_{c \to j}$, where $n_{c \to j}$ counts samples with true label c predicted as j. The goal is to update h_θ using a small label budget $B \ll |\mathcal{D}^{\text{tgt}}|$ and optional reference-period replay, recovering collapsed classes while preserving stable-class performance.

B. Longitudinal Analysis Setup

CESNET-TLS-Year22 [24] provides 12 monthly partitions of encrypted application flows. We use the DataZoo `size=S` split (178 classes), with 1,323,229 M12 target instances represented by the first 30 packet sizes, directions, and inter-packet times. A three-layer 1D CNN with GroupNorm and a 256-dimensional representation is trained on M3 and evaluated on M4–M12. Source training uses cross-entropy and three auxiliary self-supervised tasks for 30 epochs with AdamW (learning rate 10^{-3}, batch size 1024). The encoder remains frozen during repair. The separate household study evaluates device-type labels.

C. The Absorber-Collapse Pattern

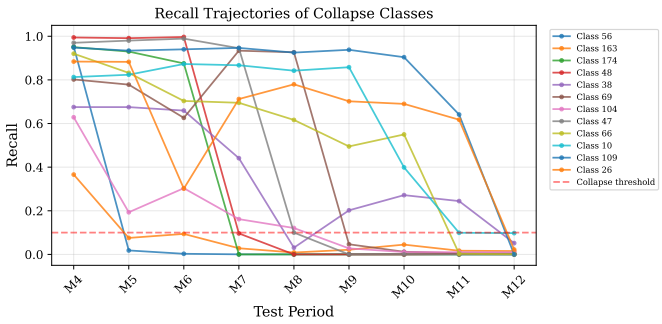


Fig. 1. Recall trajectories of 12 collapse classes on CESNET-TLS-Year22. Classes exhibit both abrupt collapse (*e.g.*, class 56 at M5) and gradual decline (*e.g.*, class 104 over M4–M12). By M12, all 12 classes fall below the 0.1 recall threshold (dashed line).

Figure 1 shows abrupt failure (class 56: recall 0.952 at M4 to 0.018 at M5) and gradual failure (class 104: 0.629 at M4 to 0.010 at M12). Each collapsed victim has a primary predicted absorber; Table I lists the 12 M12 pairs, whose dominant confusion rates range from 13.9% to 46.1%.

TABLE I
Absorber-collapse pairs at M-2022-12. Each collapsed class has a dominant confusion target (absorber). Confusion rate = fraction of victim samples predicted as the absorber.

Victim Class	Absorber Class	Confusion Rate	Victim Recall	Pattern Type	First Collapse
56	96	25.2%	0.000	abrupt	M5
163	46	41.8%	0.015	gradual	M5
174	2	29.2%	0.000	abrupt	M7
48	14	39.0%	0.001	abrupt	M7
38	45	46.1%	0.052	abrupt	M8
69	105	37.7%	0.005	abrupt	M9
104	2	21.2%	0.010	gradual	M9
47	5	45.1%	0.000	gradual	M11
66	71	13.9%	0.001	abrupt	M11
10	156	36.5%	0.098	gradual	M11
109	71	24.9%	0.000	abrupt	M12
26	13	21.6%	0.022	abrupt	M12

D. Two Empirical Signatures of Collapse

We compare victim–absorber Fisher discriminant ratios at M4 and M12. Nine pairs show reduced separability (relative decreases of 14–59%); three show increased separation (30–45%) despite continued absorption by the frozen head. The latter is consistent with a boundary mismatch. These descriptive signatures suggest that target-label boundary repair and broad class preservation are both relevant, but do not identify causal shift mechanisms.

E. Why General TTA Methods Fail

We adapt Tent, EATA, SAR, CoTTA, and NOTE to 1D packet sequences; EATA, CoTTA, and NOTE require simplified traffic-specific components. All five leave M12 collapse F1 below 0.031 in Table II, including across five learning rates from 10^{-5} to 10^{-3}. When the model confidently predicts an absorber for a victim, minimizing entropy supplies no signal for the correct victim class. The implementation adaptations limit conclusions about the original methods, but the shared negative result motivates verified target labels.

IV. Method: Collapse-Aware Repair

CARE combines an unlabeled class-health monitor, active target-label acquisition, and head repair with all-class replay and distillation (Algorithm 1). Figure 2 places these steps in a proposed operator-reviewed traffic-analytics workflow. The household experiment in Section V-D4 evaluates repair on device types; it does not implement a gateway.

A. Unsupervised Collapse Detection and Monitoring

For each class c, the monitor compares reference and target predictions using five unlabeled signals: predicted-class count ratio $r_c = (n_c^{\text{tgt}}/N^{\text{tgt}})/(n_c^{\text{ref}}/N^{\text{ref}})$, Fréchet feature distance, mean margin drop, mean confidence drop, and mean entropy rise. The score is

$$s_c = 0.40 \max(0, 1 - r_c) + 0.20\tilde{d}_c + 0.15\tilde{m}_c + 0.15\tilde{f}_c + 0.10\tilde{e}_c, \quad (1)$$

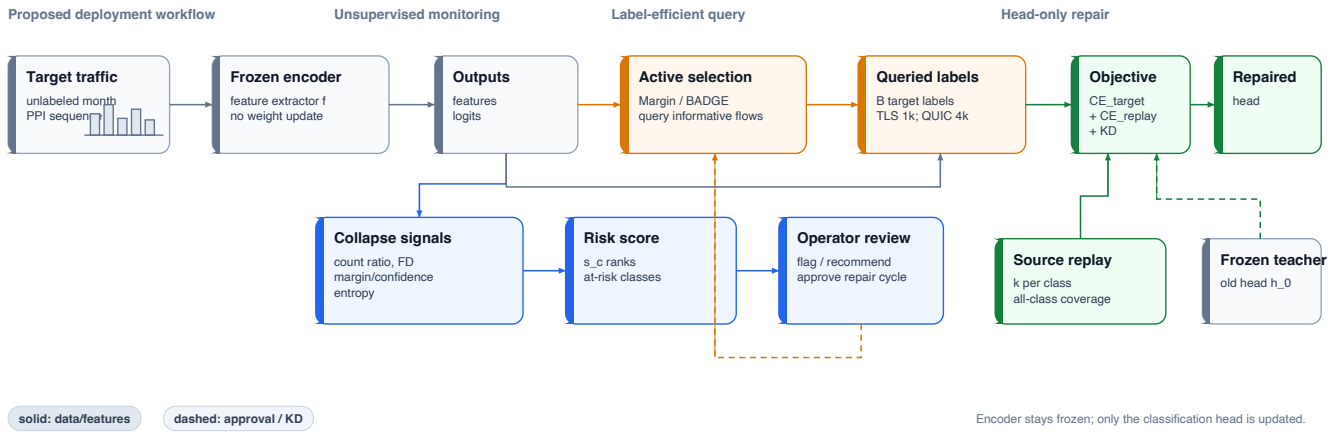


Fig. 2. Proposed operator-reviewed CARE workflow. A frozen encoder produces target-period logits and features. Monitoring ranks class-level risk and flags classes for operator review; only an approved repair cycle proceeds to active label selection and offline head-only repair with all-class reference-period replay and knowledge distillation.

where tildes denote median-absolute-deviation-normalized signals. We flag scores above 0.12, retaining at most 20 classes for operator review. Weights and threshold are fixed heuristics; Section V-E reports diagnostic performance. The monitor does not select replay classes, which always cover the full label space.

B. Active Sample Selection

We select B target flows for labeling. Margin ranks globally small top-two logit gaps; BADGE [20] combines uncertainty and gradient-embedding diversity. Our BADGE implementation uses top-three-class gradient embeddings, 50-dimensional principal component analysis, and B k-means++ seeding steps. Both selectors use the same subsequent repair procedure.

C. Head Fine-Tuning with All-Class Reference-Period Replay

We precompute frozen features $z = f(x)$ and update only the head h_θ. The training multiset contains ρ copies of each queried target feature and $k = 5$ labeled reference features per class (890 total for TLS; default $\rho = 2$). Its cross-entropy loss is

$$\mathcal{L}_{\text{ft}} = |D^z_{\text{train}}|^{-1} \sum_{(z,y)\in D^z_{\text{train}}} \ell_{\text{CE}}(h_\theta(z), y). \tag{2}$$

At $B = 1{,}000$, this uses 1,000 newly labeled target flows and 2,890 training entries after repetition and replay.

D. Knowledge Distillation

On replay feature exemplars, we add a KL-divergence loss [25] between the updated head h_θ and original head h_{θ_0}:

$$\mathcal{L}_{\text{KD}} = \frac{T^2}{|D^z_{\text{replay}}|} \sum_{z\in D^z_{\text{replay}}} D_{\text{KL}}\left(\sigma\left(\frac{h_{\theta_0}(z)}{T}\right) \middle\| \sigma\left(\frac{h_\theta(z)}{T}\right)\right) \tag{3}$$

where T is the temperature and σ is softmax. The T^2 factor matches the implementation. Each update independently samples a reference minibatch with replacement for distillation. Distillation is applied only on replay features because the teacher's reference-period predictions represent its source behavior. On target samples, the teacher's predictions may already reflect absorber-collapse errors, making them unsuitable as distillation targets.

Algorithm 1 CARE: Collapse-Aware Repair

Require: Source model f, h_{θ_0}; labeled reference features and reference predictions; unlabeled target data D_{tgt}; budget B; repeat ρ; epochs E

Monitor: Compute per-class risk scores s_c (§IV-A) from unlabeled predictions
Alert: Flag risk for operator review; proceed only after operator approval
Compute features and logits for D_{tgt}
Select B samples via margin or BADGE; query labels $\rightarrow D^z_{\text{tgt}}$
Sample D^z_{replay}: k=5 frozen features from labeled reference-period data for all 178 classes
Build D^z_{train} with ρ target copies and one reference copy
Initialize $h_\theta \leftarrow h_{\theta_0}$
for epoch $= 1, \ldots, E$ **do**
  for minibatch in a shuffled pass over D^z_{train} **do**
    Independently sample a reference minibatch for KD
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{ft}} + \lambda_{\text{KD}} \cdot \mathcal{L}_{\text{KD}}$
    Update θ via AdamW on $\mathcal{L}$
  end for
end for
return Updated classifier h_θ

E. Combined Objective and Algorithm

The full training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{ft}} + \lambda_{\text{KD}} \cdot \mathcal{L}_{\text{KD}} \tag{4}$$

V. Experiments

This section evaluates CARE through overall repair performance and baseline comparisons, internal mechanisms, cross-setting stress tests, and diagnostic detection quality. We first describe the primary datasets, evaluation protocols, metrics, and implementation settings; the distinct IoT household protocol is specified with its results.

A. Experimental Setup

Datasets and protocol. CESNET-TLS-Year22 [24] is the primary 178-class benchmark (M3 source; M7, M9, M11, M12 targets). CESNET-QUICEXT-25 XS [26], [27] provides a 44-class cross-protocol test (M-2024-6 to M-2025-5). The TLS dataset also contains a documented flow-exporter change near week 10 [24]; our evaluation measures the combined temporal shift. In the main *strict* protocol, queried target flows are excluded from evaluation. A separate seed-matched protocol holds out 20% before adaptation. Static/TTA rows use one fixed source checkpoint; active rows average five repair seeds. Other seed counts are stated where used.

Comparisons and metrics. We test five unlabeled TTA variants, a Chen-inspired pseudo-label baseline [12], entropy/coreset/random selectors, and adapted continual-learning methods. Labeled comparisons use the same $B = 1{,}000$ budget unless stated otherwise. We report macro-F1; F1 of the 12 M12 classes with static recall < 0.1 (Col.); F1 of a fixed 20-class M4–M12 stable diagnostic set (Stb.); and the number of those 12 classes still below 0.1 recall (#Col.). Stable classes are selected retrospectively by the smallest absolute F1 change among classes with ≥ 100 M12 instances and M4 support; this is a preservation endpoint, not a prospective detection set. Per-class recovery is full at recall ≥ 0.30, partial at 0.05–0.30, and failed below 0.05.

Source training and exclusion. The 1D CNN uses three convolutional layers with GroupNorm and a 256-dimensional hidden representation. It is trained for 30 epochs with AdamW; the selected source checkpoint is fixed across primary repair seeds. Query IDs are removed before strict target evaluation, and the five seeds repeat repair rather than source-model training. The 20-class stable endpoint is reported separately from the 12-class collapse endpoint so that recovery does not hide collateral damage.

CARE settings. Unless specified, $B = 1{,}000$, five M4 reference features per class, target repeat $\rho = 2$, distillation weight $\lambda_{\mathrm{KD}} = 0.5$, temperature $T = 2$, and 30 head-update epochs with learning rate 10^{-3}.

B. Overall Performance

1) Primary Results under Strict Query-Excluded Evaluation: Table II compares methods nine months after TLS source training.

CARE-BADGE reaches 0.700 ± 0.003 macro-F1 and 0.332 ± 0.040 collapse F1, compared with static values 0.629 and 0.028; its stable-class F1 is 0.894 versus 0.903 static. Margin reaches 0.686 macro-F1 and 0.256 collapse F1 at lower selection cost. The five TTA methods leave collapse F1 below 0.031. Paired t-tests favor Margin over Random and BADGE over Margin, but with five seeds exact two-sided Wilcoxon tests cannot resolve either difference at 0.05.

TABLE II
Main results on CESNET-TLS-Year22 at M-2022-12. Static/TTA rows are single-checkpoint evaluations without labels. Active rows use B=1,000 labels, M4 all-class replay (k=5, 890 features), KD (λ=0.5), strict query exclusion, and five seeds.

Method	Macro	Col. F1	Stb. F1	#Col
No labeled target data:				
Static	0.629	0.028	0.903	12
Tent	0.631	0.026	0.903	12
EATA	0.629	0.028	0.903	12
SAR	0.629	0.026	0.901	11
CoTTA	0.630	0.027	0.903	12
NOTE	0.630	0.027	0.902	12
Active (B=1000, all-class replay+KD, 5 seeds):				
Entropy	0.628±0.004	0.078±0.015	0.887±0.004	10.2
Coreset	0.633±0.005	0.116±0.042	0.886±0.007	8.8
Random	0.668±0.004	0.188±0.024	0.892±0.006	8.2
CARE-Margin	0.686±0.001	0.256±0.009	0.892±0.003	5.8
CARE-BADGE	**0.700±0.003**	**0.332±0.040**	**0.894±0.004**	**4.0**

2) Matched Holdout and Self-Evolving Baseline: In a separate five-seed experiment, 20% of the M12 pool is held out before adaptation. A Chen-inspired full-model pseudo-label baseline [12] uses no human target labels; a $p \geq 0.997$ confidence setting reaches 0.505 macro-F1 and 0.001 collapse F1, while the post-hoc best-macro $p \geq 0.90$ sensitivity setting reaches 0.614 and 0.014. CARE-BADGE, with 1,000 verified labels and head-only repair, reaches 0.701 ± 0.007 and 0.310 ± 0.047 on identical seed-wise holdouts. The supervision and update budgets differ, so this is a practical comparison rather than an isolated component effect. Table III reports the full matched comparison and program wall time; S1 supplies the archived records.

3) Comparison with Continual Learning Baselines: We adapt MEMENTO [15] (full-model update with replay and KD), ILETC [16] (generated or real replay without KD), CADE-score [21] (drift-score query selection), and an expanded head inspired by PRIME [17]. All use the strict protocol, five seeds, and 1,000 labels; other than CADE-score and CARE-BADGE, they share Margin queries. The adaptations are task-specific implementations, not exact reproductions.

Table IV shows that CARE-BADGE has the highest mean macro-F1 and collapse F1 among these rows. MEMENTO reaches 0.678/0.255; the expanded head reaches 0.673/0.307 but has lower stable F1 (0.871). ILETC variants retain less stable-class performance (0.851–0.854). These contrasts support the bounded head-update and preservation design in this setting, without isolating every architectural or loss difference.

4) Multi-Period Generalization: Table V evaluates Margin-based repair at four drift periods.

Macro-F1 and fixed-group collapse F1 improve at all four periods. The M12 macro-F1 gain is 0.057; earlier intervention at M7 yields 0.633 collapse F1. These are independent monthly repairs, not a sequential deployment.

C. Internal Mechanism Analysis

1) Component-Variant Study: Table VI compares seven evaluated bundles under Margin queries and five seeds. It is

TABLE III
SEED-MATCHED 20% HOLDOUT COMPARISON AT M12 (MEAN ± SAMPLE STANDARD DEVIATION OVER 5 SEEDS). HUMAN LABELS COUNT UNIQUE QUERIED TARGET LABELS; PSEUDO-LABELS ARE THE MEAN HIGH-CONFIDENCE TARGET SAMPLES USED FOR OUR CHEN-STYLE FULL-MODEL FINE-TUNING. PROGRAM WALL TIME IS MEASURED ON ONE NVIDIA L20 UNDER EACH METHOD'S FINAL CONFIGURATION; DASHES DENOTE CONFIGURATIONS NOT INCLUDED IN THE TIMING RUN.

Method	Human	Pseudo	Macro F1	Collapse F1	Stable F1	#Col.	Wall (s)
Static	0	0	0.6308±0.0020	0.0313±0.0055	0.9019±0.0015	11.8	–
Chen-style ($p \geq 0.997$)	0	298.8K	0.5050±0.0097	0.0008±0.0018	0.8063±0.0208	12.0	80.0±1.0
Chen-style ($p \geq 0.90$, sensitivity)	0	628.7K	0.6140±0.0058	0.0136±0.0035	0.8887±0.0080	12.0	136.4±0.5
CARE-Entropy	1,000	0	0.6316±0.0025	0.0797±0.0097	0.8863±0.0039	10.0	–
CARE-Random	1,000	0	0.6722±0.0021	0.2246±0.0370	0.8934±0.0039	7.0	–
CARE-Margin	1,000	0	0.6885±0.0022	0.2835±0.0185	0.8932±0.0047	**4.6**	48.5±0.8
CARE-BADGE	1,000	0	**0.7008±0.0068**	**0.3099±0.0473**	**0.9004±0.0042**	4.8	166.6±8.2

TABLE IV
CONTINUAL-LEARNING AND DRIFT-ADAPTATION BASELINES AT M-2022-12 (B=1,000, STRICT EVAL, 5 SEEDS). CADE-SCORE AND CARE-BADGE USE THEIR NAMED SELECTORS; OTHER LABELED ROWS USE MARGIN. CARE ROWS ARE REPRODUCED FROM TABLE II.

Method	Macro	Col. F1	Stb. F1	#Col
Static	0.629	0.028	**0.903**	12.0
CL / drift baselines (same label budget):				
CADE-score [21]	0.637±0.004	0.118±0.015	0.889±0.003	9.0
ILETC-GAN [16]	0.650±0.004	0.219±0.010	0.851±0.009	5.4
ILETC-real	0.659±0.003	0.236±0.010	0.854±0.016	4.8
Expanded head	0.673±0.003	0.307±0.005	0.871±0.008	4.8
MEMENTO [15]	0.678±0.002	0.255±0.005	0.892±0.004	6.0
CARE-Margin	0.686±0.001	0.256±0.009	0.892±0.003	5.8
CARE-BADGE	**0.700±0.003**	**0.332±0.040**	0.894±0.004	**4.0**

TABLE V
MULTI-PERIOD RESULTS (MARGIN, B=1000, ALL-CLASS REPLAY k=5, 3 SEEDS). THE FIXED COLLAPSE SET IS THE 12 CLASSES IDENTIFIED BY THE M12 STATIC MODEL.

	Static		CARE		
Period	Macro	Col.	Macro	Col.	Δ Macro
M7 (light)	0.740	0.503	0.776±0.003	0.633±0.028	+0.036
M9 (medium)	0.691	0.280	0.739±0.003	0.392±0.005	+0.048
M11 (heavy)	0.649	0.145	0.708±0.002	0.408±0.016	+0.060
M12 (severe)	0.629	0.028	0.686±0.002	0.255±0.012	+0.057

not factorial: repeat factors and replay coverage vary, so its rows do not isolate individual loss effects.

Without target labels, replay variants leave collapse F1 below 0.05. Target-only fine-tuning reaches 0.134 but reduces stable F1 to 0.839; the full Margin-based configuration reaches 0.256 collapse F1 and 0.892 stable F1. Thus target supervision is associated with recovery and reference-based training with preservation across the tested bundles. Separate matched-update reconstruction controls (S2) qualify the incremental value of reference cross-entropy. Full-model fine-tuning under the same label budget gives 0.678 macro-F1 and 0.256 collapse F1 versus 0.686 and 0.256 for head-only repair.

2) Selection Strategy Comparison: BADGE attains higher mean collapse F1 than Margin (0.332 versus 0.256) at about 3.4 times the program wall time (167 versus 49 s on one L20). With a fixed selector, Margin yields 4 fully recovered, 3 partially recovered, and 5 failed classes; BADGE yields 5, 5, and 2, respectively. Table VII shows that many recovered classes receive very few directly labeled examples, consistent with broader boundary updates rather than a positive example for every victim.

Figure 3 quantifies the label-efficiency trade-off. At $B = 2{,}000$ Margin-selected target labels, collapse F1 rises to 0.437 ± 0.004 and the number of collapsed classes falls from 12 to 2; at $B = 8{,}000$, collapse F1 reaches 0.609 ± 0.007 while stable-class F1 remains at least 0.887. The full numeric records are provided in S1. All main labeled-method comparisons use $B = 1{,}000$.

3) Replay Strategy Comparison: Table VIII compares matched-budget replay strategies.

All-class replay with five features per class yields 0.686 macro-F1. At a matched feature count, targeted replay (about 900 features) yields 0.678; all-class replay with one feature per class (178 total) yields 0.680. These comparisons favor breadth of class coverage over concentrating replay on selected classes. The detector therefore ranks classes for review, not for replay selection. A smaller targeted prototype-replay study retained 98.7% of exemplar-replay macro-F1 while reducing storage from about 150 to 60 KiB.

D. Generalization and Robustness

1) Architecture Robustness: Replacing GroupNorm with BatchNorm does not remove the collapse problem: BN-Adapt [28] leaves collapse F1 at 0.032. A targeted-replay CARE-Margin variant raises BN macro/collapse F1 from 0.613/0.030 to 0.665/0.314. A four-layer Transformer [29] with the same targeted protocol reaches 0.767/0.401. These architecture controls use a different replay variant from the default CNN experiment.

2) Cross-Protocol Generalization: QUICEXT-25 stress test.

CESNET-QUICEXT-25 XS [26], [27] provides 44 classes with M-2024-6 source and M-2025-5 target partitions. Three classes (17, 23, 25) newly fall below 0.1 recall; class 21 is excluded because it already fails in the reference period. Table IX reports strict query-excluded results across label budgets.

With 1,000 labels, CARE-Margin raises macro/collapse F1 from 0.502/0.049 to 0.564/0.114. BADGE at 4,000 labels reaches 0.586/0.191. The higher QUIC label requirement motivates protocol-specific budget choice.

TABLE VI
COMPONENT VARIANTS AT M12 (MARGIN, STRICT EVALUATION, 5 SEEDS). TARGETED-150 CONTAINS STABLE+ABSORBER FEATURES; ALL-890 CONTAINS FIVE FEATURES PER CLASS. FT, RPL, KD, AND REP. DENOTE FINE-TUNING, REPLAY, KNOWLEDGE DISTILLATION, AND TARGET REPEAT.

Config.	Target B	Replay	KD	Rep.	Macro	Col. F1	Stb. F1	#Col
Static	0	None	–	–	0.629	0.028	**0.903**	12
Replay only	0	All-890	–	–	0.563±0.007	0.044±0.019	0.816±0.026	10.2
Rpl+KD	0	All-890	✓	–	0.618±0.005	0.039±0.016	0.888±0.009	10.8
FT only	1000	None	–	1	0.638±0.000	0.134±0.001	0.839±0.001	7.0
FT+KD	1000	Targeted-150	✓	1	0.670±0.000	0.152±0.007	0.887±0.004	7.8
FT+Rpl	1000	All-890	–	2	0.659±0.003	0.236±0.010	0.854±0.016	4.8
Full CARE	1000	All-890	✓	2	**0.686±0.001**	**0.256±0.009**	0.892±0.003	5.8

TABLE VII
PER-CLASS RECOVERY AT M-2022-12 (B=1,000, ALL-CLASS REPLAY k=5, STRICT EVAL, 5 SEEDS; MEAN ± SAMPLE STANDARD DEVIATION). SELECTED = MEAN TARGET LABELS ALLOCATED TO THAT CLASS.

			Static		CARE-Margin		CARE-BADGE		Selected	
Class	Abs.	n	Recall	F1	Recall	F1	Recall	F1	Marg	BADGE
56	96	12137	0.000	0.001	0.722±0.033	0.784±0.008	**0.793±0.017**	**0.829±0.021**	12.0	13.6
163	46	1741	0.016	0.026	0.143±0.031	0.168±0.024	**0.226±0.140**	**0.235±0.123**	3.0	4.8
174	2	1507	0.000	0.000	**0.195±0.005**	**0.262±0.012**	0.029±0.040	0.044±0.060	4.0	1.6
48	14	3361	0.001	0.002	0.437±0.009	0.568±0.007	**0.722±0.234**	**0.767±0.146**	4.0	4.8
38	45	193	0.052	0.090	**0.656±0.095**	**0.410±0.057**	0.242±0.277	0.210±0.210	3.0	1.2
69	105	1866	0.005	0.008	0.031±0.017	0.042±0.021	**0.157±0.203**	**0.177±0.219**	3.0	2.4
104	2	1744	0.010	0.016	**0.703±0.044**	**0.664±0.031**	0.604±0.154	0.594±0.073	8.0	3.6
47	5	618	0.000	0.000	0.002±0.002	0.004±0.003	**0.404±0.201**	**0.462±0.168**	0.0	2.2
66	71	1955	0.001	0.002	0.001±0.000	0.002±0.001	**0.133±0.118**	**0.144±0.121**	1.0	4.2
10	156	1004	0.098	0.162	0.074±0.030	0.121±0.044	**0.389±0.215**	**0.441±0.193**	0.0	1.8
109	71	393	0.000	0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.0	1.6
26	13	807	0.022	0.025	0.035±0.012	0.046±0.011	**0.125±0.140**	**0.087±0.069**	1.0	2.6
Macro (12 classes)			0.017	0.028	0.250	0.256	**0.319**	**0.332**		

TABLE VIII
REPLAY STRATEGY COMPARISON (MARGIN, B=1000, 5 SEEDS). "TARGETED" USES AN ORACLE STABLE+ABSORBER LIST; RESULTS SUPPORT CLASS-COVERAGE BREADTH RATHER THAN REPLAY VOLUME AS THE EXPLANATION IN THIS COMPARISON.

Replay Config.	Macro	Col. F1	n_{rpl}
All-class k=5 (890)	**0.686±0.001**	**0.256±0.009**	890
Targeted k=30 (∼900)	0.678±0.001	0.243±0.003	∼900
All-class k=1 (178)	0.680±0.003	0.231±0.015	178

TABLE IX
CROSS-PROTOCOL GENERALIZATION ON CESNET-QUICEXT-25 XS (M-2024-6 → M-2025-5, 44 CLASSES, 5 SEEDS). B IS THE NUMBER OF QUERIED TARGET LABELS; STRICT EVALUATION EXCLUDES THEM.

Method	B	Macro-F1	Collapse F1	#Col.
Static	0	0.502	0.049	3.0
CARE-Margin	1000	0.564±0.003	0.114±0.002	2.0±0.0
CARE-BADGE	1000	0.563±0.006	0.091±0.017	2.8±0.45
CARE-Margin	2000	0.577±0.003	0.126±0.014	2.0±0.0
CARE-BADGE	2000	0.578±0.006	0.144±0.031	2.2±0.84
CARE-Margin	4000	0.578±0.002	0.132±0.021	1.8±0.45
CARE-BADGE	4000	**0.586±0.007**	**0.191±0.030**	**1.2±0.45**

3) Training-Seed and Checkpoint Robustness: A targeted-replay retraining audit (3×3=9 runs) yields static macro-F1 of 0.620±0.001 and CARE macro-F1 of 0.667±0.007 across training seeds. Our primary checkpoint (static 0.629, best epoch 27/30, validation F1 0.892) is about 0.009 above the audit mean, larger than the audit standard deviation and therefore evidence of checkpoint/run sensitivity. The targeted-replay CARE variant improves over its corresponding static

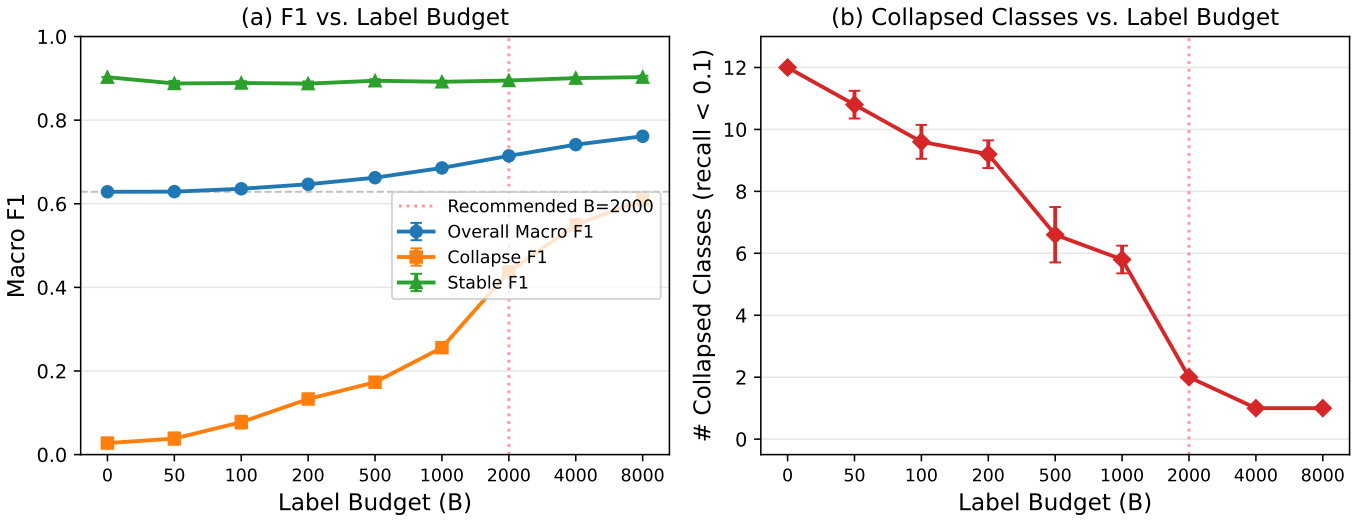


Fig. 3. Label budget vs. performance (margin, 5 seeds, error bars). Left: Overall, collapse, and stable macro F1. Right: number of collapsed classes (recall < 0.1). The knee at B=2,000 reduces collapsed classes from 12 to 2; beyond B=4,000 gains are marginal.

model in all audited training seeds, although the gain magnitude varies.

4) Exploratory IoT Device-Type Stress Test: We evaluate a public residential IoT flow archive [1] as a closed-set, mixed-protocol device-type task. The official MAC inventory provides 24 type labels; MAC/IP addresses, ports, timestamps, and labels are excluded from model inputs, which comprise 22 flow statistics and six protocol indicators. Eleven homes contain all 24 source types. Each home uses chronological source, query, development, and final windows split at approximately 50%, 70%, and 85% of observed days, with a separate 28-input tabular MLP. For each home and five overlapping repair seeds, 1,000 uniformly sampled query labels and five reference features per class train the ported CARE head. This port differs from the CNN/BADGE main study and evaluates repair without monitoring.

Source training uses a class-capped reservoir (up to 5,000 flows per type); the query pool is a uniform reservoir from the subsequent chronological window. The model uses flow statistics and protocol indicators rather than device identifiers, so the IoT experiment tests traffic-based type recognition rather than memorization of MAC addresses. The five repair seeds share each home's source model and are repeated measurements, not additional homes.

The IoT source head is trained on a 28–256–128–24 MLP for 30 epochs. Its repair uses 500 head-update steps with the same 1,000 paid labels for CARE and target-only updating; CARE additionally uses five historical features per class and the reference losses. The interpolation control combines frozen source and target-only head weights at a fixed 0.5 ratio. Accordingly, these controls compare maintenance choices under the same paid-label budget, but the household protocol is not a direct reproduction of the TLS classifier or BADGE query policy.

Development analyses informed a fixed final comparison of static, target-only, CARE, and standard 0.5 source–target head interpolation (BCWI). Refit predictions were checked against frozen development outputs; final-window predictions and scoring rules were frozen before final labels were decoded. Table X reports the final per-home results; homes are independent units.

On development windows, CARE improves equal-home macro-F1 from 0.54279 to 0.54424 (6/11 homes). Among 17 low-recall home–type pairs with at least 50 rows, all improve in mean F1 and 11 reach recall at least 0.1; 12 of the 17 receive fewer than five paid positives on average. Among 75 initially high-recall pairs, seven lose more than five F1 points. These outcomes motivated explicit final-window class-level endpoints, without changing CARE training.

On final windows, low-recall-class F1 rises from 0.03988 (static) to 0.13887 (CARE), while equal-home macro-F1 changes from 0.52006 to 0.51239 (-0.767 percentage points; five homes improve). The descriptive home-bootstrap interval for the mean difference is $[-2.474, 0.756]$ points. CARE's conditional negative-flip rate is 0.05908; it restores recall to at least 0.1 for an average 0.78 of 1.64 low-recall types per home, while 0.42 of 5.55 stable types per home lose more than five F1 points. Table XI places this recovery–preservation trade-off beside the competitive BCWI control: BCWI has the highest mean home-level F1 and fewer negative flips than CARE, while CARE recovers more low-recall types and has the highest mean F1 for that group. Target-only updating is weaker on both whole-model F1 and negative flips. No pair meets the stricter development-to-final incident-collapse definition. Preselected home-level shift indicators show no clear association with CARE benefit. This stress test demonstrates device-type recovery alongside whole-model variation and harm in some homes.

The per-home spread is material: CARE gains 2.84 and 2.05 F1 points in homes 08 and 01 but loses 6.41 and 5.27 points in homes 06 and 07. These are paired home-level outcomes, not independent seed-level replications. A descriptive analysis of development-to-final class-share distance, largest final class share, and paid-positive coverage among low-recall types gives

TABLE X
IoT DEVICE-TYPE FINAL-WINDOW HOLDOUT ON 11 HOMES. EACH HOME USES A SEPARATE SOURCE MLP AND 1,000 UNIFORM PAID LABELS PER REPAIR SEED; CARE AND THE FIXED SOURCE–TARGET HEAD-INTERPOLATION CONTROL (BCWI 0.50) AVERAGE FIVE OVERLAPPING SEEDS. MACRO-F1 AVERAGES DEVICE TYPES PRESENT IN THAT HOME'S FINAL WINDOW. DELTAS ARE PERCENTAGE POINTS VERSUS THE STATIC MODEL, COMPUTED BEFORE ROUNDING. HOMES, RATHER THAN SEEDS, ARE INDEPENDENT UNITS.

Home	Static F1	CARE F1	Δ_{CARE} (pp)	BCWI F1	Δ_{BCWI} (pp)
01	0.5895	0.6100	$+2.047$	0.6106	$+2.106$
02	0.4623	0.4606	-0.172	0.4915	$+2.916$
03	0.5201	0.5171	-0.306	0.5421	$+2.198$
04	0.4327	0.4444	$+1.163$	0.4539	$+2.115$
05	0.4498	0.4332	-1.661	0.4636	$+1.386$
06	0.5470	0.4829	-6.410	0.5109	-3.615
07	0.5536	0.5009	-5.272	0.4603	-9.331
08	0.4930	0.5214	$+2.838$	0.5429	$+4.987$
09	0.5564	0.5687	$+1.234$	0.6095	$+5.310$
11	0.5553	0.5593	$+0.399$	0.5511	-0.420
12	0.5609	0.5379	-2.293	0.5780	$+1.716$
Equal-home mean	0.52006	0.51239	-0.767	0.52858	$+0.852$

TABLE XI
PRESPECIFIED IoT FINAL-WINDOW CLASS-LEVEL OUTCOMES, EQUAL-HOME MEANS OVER 11 HOMES. LOW-RECALL TYPES HAVE FINAL SUPPORT ≥ 50 AND STATIC FINAL RECALL < 0.1 (MEAN 1.64 TYPES/HOME); STABLE TYPES HAVE DEVELOPMENT AND FINAL SUPPORT ≥ 50 AND STATIC RECALL ≥ 0.8 IN BOTH WINDOWS (MEAN 5.55 TYPES/HOME). RECOVERY COUNTS LOW-RECALL TYPES WHOSE CHALLENGER RECALL REACHES ≥ 0.1; STABLE LOSSES COUNT F1 DROPS > 0.05 RELATIVE TO STATIC. NEGATIVE-FLIP RATE IS CONDITIONAL ON STATIC-CORRECT FLOWS. FIVE REPAIR SEEDS WITHIN EACH HOME OVERLAP.

Method	Macro-F1	Low-recall F1	Recovered types/home	Stable F1 Δ	Stable losses/home	Negative flips
Static	0.52006	0.03988	0.00	0.00000	0.00	0.00000
Target-only	0.48581	0.10343	0.55	-0.00059	0.73	0.12315
CARE	0.51239	**0.13887**	**0.78**	**0.01932**	0.42	0.05908
BCWI 0.50	**0.52858**	0.10379	0.49	0.01801	0.29	0.04756

Spearman correlations of 0.027, -0.345, and -0.012 with CARE's home-level F1 change, respectively. None identifies a simple household condition under which the same update is reliably beneficial. The paired class-level table is therefore more informative for maintenance decisions than the mean gain alone.

E. Diagnostic Detector Evaluation

At M12, the five-signal score detects 10 of 12 collapsed classes at precision 0.50 and recall 0.833; count drop plus Fréchet distance alone gives 0.45/0.75. Table XII shows the monitor across drift periods.

TABLE XII
DETECTION ACROSS DRIFT LEVELS (k=20, RECALL<0.1, SUPPORT$\geq$50). GT DENOTES GROUND TRUTH; RECALL IS $\geq$0.80 THROUGHOUT.

Period	GT	Precision	Recall	F1
M7 (light)	5	0.200	0.800	0.320
M9 (medium)	8	0.350	0.875	0.500
M11 (heavy)	10	0.450	0.900	0.600
M12 (severe)	12	0.500	0.833	0.625

Detection recall stays at least 0.80 across periods, while precision rises from 0.20 at M7 to 0.50 at M12. Equal and historically calibrated weight variants give M12 detection F1 between 0.563 and 0.625. These are retrospective diagnostic operating points, not an automatic update gate.

VI. DISCUSSION

A. Class-Level Maintenance

The TLS experiments show that verified target labels can recover classes missed by confidence-based adaptation, while reference replay and distillation limit damage to stable classes. CARE-BADGE offers the strongest mean recovery in the primary comparison; Margin has lower selection cost. The QUIC stress test shows the same direction with a larger budget. The IoT holdout exposes the harder acceptance problem: improved recognition of some device types can coexist with losses in other types and homes. A candidate head should therefore be assessed with per-class recovery and harm measures before activation.

The component-variant results also clarify the scope of the claim. Target-label supervision supplies a correction signal absent from the tested unlabeled methods, whereas all-class replay and source-head distillation constrain changes to previously learned classes. Because some component variants alter repeat factors and replay coverage together, they support the combined maintenance design rather than a causal ranking of every loss term. The separately reconstructed controls in S2 further qualify the incremental value of reference cross-entropy. These distinctions matter when porting the method: the IoT device-type update retains the head-repair principle but uses a different feature space, label distribution, and query policy.

B. Operational Scope and Limitations

Figure 2 depicts an operator-reviewed cycle: monitor unlabeled traffic, inspect class-risk alerts, acquire limited labels, repair offline, and validate a candidate head while the deployed model continues serving. The evaluated TLS cycle uses 1,000 target labels and 890 stored reference features; human review and deployment costs were not measured. The detector prioritizes recall and incurs false alerts, so it is a screening tool. Head-only repair also depends on a usable frozen representation and cannot add unseen classes.

For a real traffic-analytics service, acceptance should be based on the classes that motivated maintenance and on the classes already working well. The operator could compare the candidate head with the retained source head on a reviewed validation batch, inspect newly missed classes and negative flips, and deploy only after those outcomes meet local error costs. This is a proposed workflow rather than a tested online gate. The monthly TLS analysis gives one maintenance interval, while shorter windows or automatic scheduling would need separate validation.

The monitored class-health score is a proposal for human review. Its M12 precision of 0.50 means that even a sensitive alert cannot by itself justify deployment. At a candidate update, the operator can compare per-class confusion and negative flips against the existing model; the IoT holdout shows why this check matters even when a low-recall category recovers. A live acceptance policy and its review cost remain to be measured.

The primary benchmark is university-network TLS application traffic. The household port uses known device types, mixed-protocol flow features, uniform queries, and a different update schedule. It does not test an encrypted-only IoT gateway or the full CARE monitoring pipeline. Future work should validate update acceptance across sites, annotation costs, and selective encoder updates when frozen features are inadequate.

VII. Conclusion

We characterize absorber collapse and evaluate CARE as a class-aware repair workflow. On CESNET-TLS-Year22, 1,000 verified labels raise macro-F1 from 0.629 to 0.700 and collapsed-class F1 from 0.028 to 0.332 with BADGE, while stable-class F1 remains 0.894 versus 0.903 static. QUIC results support transfer across the tested encrypted-application settings. The 11-home device-type holdout shows low-recall recovery but mixed whole-model outcomes, making per-class validation central to practical maintenance.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and Code Availability

The public CESNET-TLS-Year22 [24], CESNET-QUICEXT-25 [26], [27], and residential IoT [1] datasets are available at https://doi.org/10.5281/zenodo.10608607, https://doi.org/10.5281/zenodo.17249078, and https://iotanalytics.unsw.edu.au/homedataset.html, respectively. Supplementary Material S1 contains archived CESNET records, implementation files, available checkpoints, and result provenance; original size-S row identities and full caches are unavailable, so complete end-to-end reproduction is not possible from S1 alone. S2 reports separate reconstruction controls, which do not replace the original split. S3 provides the frozen IoT final-window protocol, predictions, tables, and file hashes; rescoring requires the public raw archive.

Acknowledgment

OpenAI Codex assisted with language editing throughout the manuscript and with figure rendering from archived numeric records in the Empirical Analysis and Experiments sections. It also assisted with code implementation and result verification for the IoT device-type holdout in the Experiments section and the separate reconstruction controls reported in Supplementary Material S2. The authors reviewed the generated text, code, figures, and results and take responsibility for the accuracy and integrity of the article.

References

[1] A. Pashamokhtari, N. Okui, M. Nakahara, A. Kubota, G. Batista, and H. Habibi Gharakheili, "Dynamic inference from IoT traffic flows under concept drifts in residential ISP networks," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15 761–15 773, 2023.

[2] S. Azizi, F. V. Calderan, N. Okui, M. Nakahara, A. Kubota, M. G. Quiles, G. Batista, and H. Habibi Gharakheili, "Towards label shift adaptation for robust IoT device identification," in *Proceedings of the 7th Joint Workshop on CPS and IoT Security and Privacy*, 2025, pp. 105–110.

[3] S. Azizi, N. Okui, M. Nakahara, A. Kubota, G. Batista, and H. Habibi Gharakheili, "Maintaining IoT device identification under concept drift via budget-aware traffic labeling," 2026, arXiv preprint arXiv:2608.15465. [Online]. Available: https://arxiv.org/abs/2608.15465

[4] D. Soukup, R. Plný, D. Vašata, and T. Čejka, "Drift-based dataset stability benchmark," arXiv preprint, 2025.

[5] N. Malekghaini, E. Akbari, M. A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "Deep learning for encrypted traffic classification in the face of data drift: An empirical study," *Computer Networks*, vol. 225, p. 109648, 2023.

[6] K. Jerabek, J. Luxemburk, R. Plny, J. Koumar, J. Pesek, and K. Hynek, "When simple model just works: Is network traffic classification in crisis?" arXiv preprint, 2025.

[7] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=uXl3bZLkr3c

[8] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 16 888–16 905. [Online]. Available: https://proceedings.mlr.press/v162/niu22a.html

[9] S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan, "Towards stable test-time adaptation in dynamic wild world," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=g2YraF75Tj

[10] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7201–7211. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Wang_Continual_Test-Time_Domain_Adaptation_CVPR_2022_paper.html

[11] T. Gong, J. Jeong, T. Kim, Y. Kim, J. Shin, and S.-J. Lee, "NOTE: Robust continual test-time adaptation against temporal correlation," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 253–27 266. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/ae6c7dbd9429b3a75c41b5fb47e57c9e-Abstract-Conference.html

[12] Z. Chen, G. Cheng, J. Li, T. Qin, Y. Zhou, and X. Luan, "Drift-oriented self-evolving encrypted traffic application classification for actual network environment," arXiv preprint, 2025.

[13] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016. [Online]. Available: https://jmlr.org/papers/v17/15-239.html

[14] M. Long, Z. Cao, J. Wang, and M. I. Jordan, "Conditional adversarial domain adaptation," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018, pp. 1640–1650. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/hash/ab88b15733f543179858600245108dd8-Abstract.html

[15] F. Cerasuolo, A. Nascita, G. Bovenzi, G. Aceto, D. Ciuonzo, A. Pescapé, and D. Rossi, "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Computer Networks*, vol. 245, p. 110374, 2024.

[16] W. Zhu, X. Ma, Y. Jin, and R. Wang, "ILETC: Incremental learning for encrypted traffic classification using generative replay and exemplar," *Computer Networks*, vol. 224, p. 109602, 2023.

[17] T. Qin, G. Cheng, Z. Chen, and Y. Zhou, "PRIME: Plasticity-robust incremental model for encrypted traffic classification in dynamic network environments," arXiv preprint, 2025.

[18] B. Settles, "Active learning literature survey," University of Wisconsin–Madison Department of Computer Sciences, Computer Sciences Technical Report 1648, 2009. [Online]. Available: https://research.cs.wisc.edu/techreports/viewreport.php?report=1648

[19] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=H1aIuk-RW

[20] J. T. Ash, C. Zhang, A. Krishnamurthy, J. Langford, and A. Agarwal, "Deep batch active learning by diverse, uncertain gradient lower bounds," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=ryghZJBKPS

[21] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2327–2344. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin

[22] W. Liu, C. Zhu, Z. Ding, H. Zhang, and Q. Liu, "Multiclass imbalanced and concept drift network traffic classification framework based on online active learning," *Engineering Applications of Artificial Intelligence*, vol. 117, p. 105607, 2023.

[23] R. Gupta, S. Liu, R. Zhang, X. Hu, X. Wang, H. Benkraouda, P. Kommaraju, P. Cao, N. Feamster, and K. Nahrstedt, "Generative active adaptation for drifting and imbalanced network intrusion detection," arXiv preprint, 2025.

[24] K. Hynek, J. Luxemburk, J. Pešek, T. Čejka, and P. Šiška, "CESNET-TLS-Year22: A year-spanning TLS network traffic dataset from backbone lines," *Scientific Data*, vol. 11, no. 1, p. 1156, 2024.

[25] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint, 2015, presented at the NIPS 2014 Deep Learning Workshop.

[26] J. Luxemburk, K. Hynek, and J. Mücke, "CESNET-QUICEXT-25: A year-long QUIC traffic dataset with EXTended attributes," Dataset, 2025, published 2 October 2025. [Online]. Available: https://zenodo.org/records/17249078

[27] J. Mücke, M. Nawrocki, R. Hiesgen, P. Sattler, J. Zirngibl, G. Carle, J. Luxemburk, T. C. Schmidt, and M. Wählisch, "Waiting for QUIC: Passive measurements to understand QUIC deployments," *Proceedings of the ACM on Networking*, vol. 3, no. CoNEXT4, pp. 1–26, 2025, article 41.

[28] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge, "Improving robustness against common corruptions by covariate shift adaptation," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 11 539–11 551. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/85690f81aadc1749175c187784afc9ee-Abstract.html

[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017, pp. 5998–6008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html